

Session 7

# Asynchronous Infrastructure & Full CD Pipeline

June 4, 2026

Tito Alvarez

TA: Abner Pérez

## Tonight's Plan

| TIME      | BLOCK                                                  |
|-----------|--------------------------------------------------------|
| 6:00–6:10 | Cold Open — project state, Delivery 4 map, both tracks |
| 6:10–6:50 | Demo 1 — Async Module + Event-Driven Compute           |
| 6:50–7:00 | Block 1.5 — KEDA for EKS teams (concept)               |
| 7:00–7:30 | Exercise 1 — Build a Reusable Queue Module             |
| 7:30–8:10 | Demo 2 — Full CD Pipeline                              |
| 8:10–8:40 | Exercise 2 — Multi-Env Layout + Environment Promotion  |

# Cold Open

Where the project stands and what tonight unlocks for Delivery 4

# Tonight Builds Both Halves of a Production Service

## Every cloud service needs two halves

The Terraform module that provisions the async infrastructure

The CD pipeline that ships every change safely across environments

## Delivery 4 — due June 21, 2026

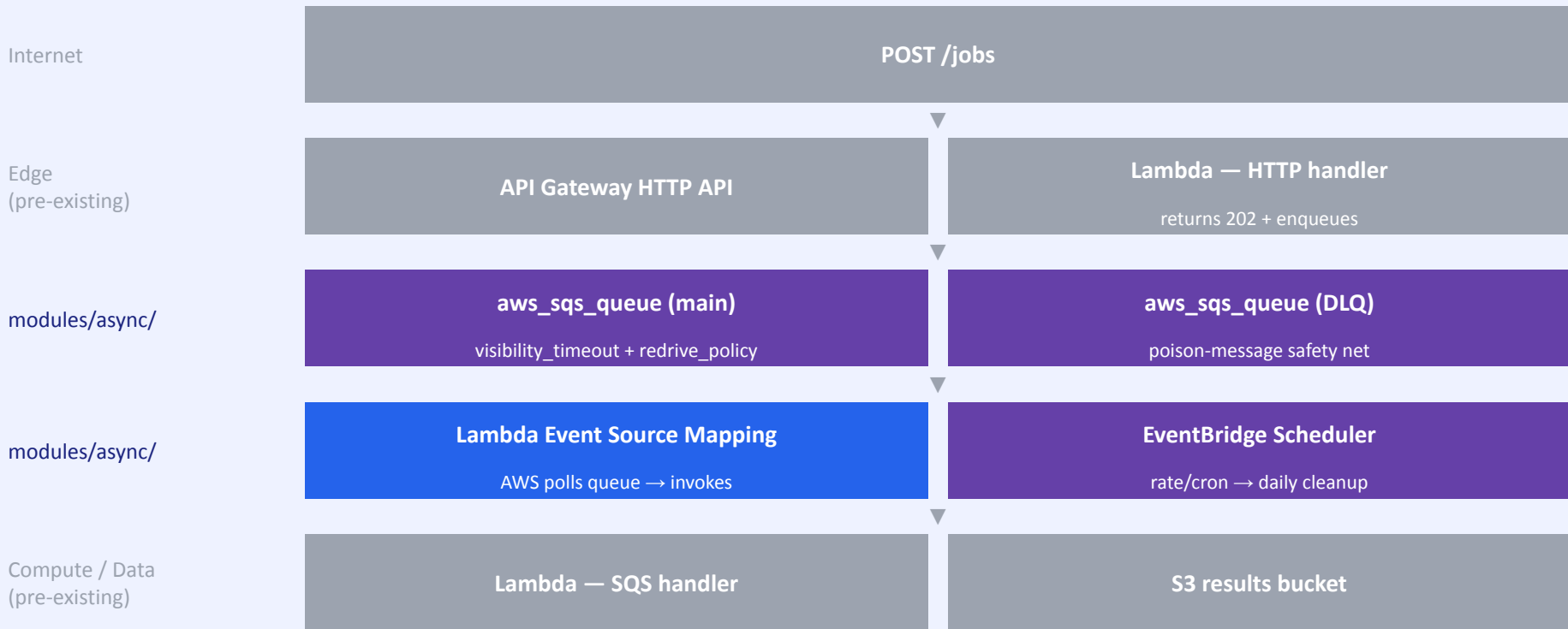
Deliverable A: reusable async module · D: plan artifact + approval gate

EKS teams: KEDA (Deliverable F) is scoped in Block 1.5

# Demo 1

Async Module + Event-Driven Compute

## Demo 1 — What We're Building



*modules/async/ outputs queue\_url / queue\_arn — the root wires a deterministic URL into the Lambda, so there is no circular module dependency.*

## Reusable Module — A Directory With a Contract

A Terraform module is just a directory containing `main.tf`, `variables.tf`, and `outputs.tf`. The module boundary is a contract: every value enters through a variable, every useful value leaves through an output, and nothing is hardcoded inside.

### The contract

Inputs: `name`, `environment`, `lambda_*`, and `tunables`

Outputs: `queue_url`, `queue_arn`, `dlq_url`, `dlq_arn`

Every variable has a description — the module's public docs

### Why it matters

Any root config calls it once to get the whole async stack

Same module serves dev, staging, prod — values differ, code doesn't

Swap the provider behind the boundary; callers don't change

## aws\_sqs\_queue — Main Queue + Dead Letter Queue

SQS decouples the HTTP request from the work. The main queue holds jobs until a consumer picks them up; a `redrive_policy` sends repeatedly-failing messages to a separate dead-letter queue instead of looping forever.

### Main queue

`visibility_timeout` hides a message while it is being processed

`redrive_policy = jsonencode({...})` references the DLQ ARN directly

`max_receive_count` delivery attempts before giving up

### Dead letter queue

Catches poison messages so they stop exhausting concurrency

`message_retention_seconds` keeps failures available for debugging

Referenced by ARN — never a hardcoded string



## aws\_lambda\_event\_source\_mapping — The Connector

The Event Source Mapping is the glue between SQS and Lambda. AWS polls the queue on your behalf and invokes the function with a batch of messages. Neither the queue nor the function knows about the other — the ESM is the only thing that links them.

### How it works

`event_source_arn` points at the main queue

`function_name` points at the consumer Lambda

`batch_size` controls messages read per invocation

### Design rule

Kept as a separate resource, not inline on the function

Tune `batch_size` or swap the source queue without touching Lambda

Both ARN references are variables — no hardcoded strings

## aws\_scheduler\_schedule — EventBridge Scheduler

EventBridge Scheduler invokes a target on a recurring rate or cron expression. Here it triggers the same Lambda with a scheduler payload so the function can run a periodic cleanup pass instead of processing a job.

### Configuration

`schedule_expression` + `scheduler_timezone` as variables

`flexible_time_window = OFF` means exact timing

input payload tags the invocation as `source = scheduler`

### Identity

A dedicated IAM role — not the Lambda execution role

`lambda:InvokeFunction` scoped to the exact function ARN

Each service gets its own scoped identity, never a wildcard

► **LIVE DEMO**

**Demo 1**

# **Async Module + Event-Driven Compute**

```
mkdir -p infra/modules/async  
touch infra/modules/async/main.tf \  
infra/modules/async/variables.tf \  
infra/modules/async/outputs.tf
```

**A module is a directory with a contract.**

```
resource "aws_sqs_queue" "dlq" {  
  name = "${var.name}-${var.environment}-dlq"  
  message_retention_seconds = var.dlq_message_retention_seconds  
}
```

**The DLQ is your poison-message safety net.**

```
resource "aws_sqs_queue" "main" {  
  name = "${var.name}-${var.environment}-queue"  
  visibility_timeout_seconds = var.visibility_timeout_seconds  
  
  redrive_policy = jsonencode({  
    deadLetterTargetArn = aws_sqs_queue.dlq.arn  
    maxReceiveCount     = var.max_receive_count  
  })  
}
```

**`redrive_policy` points at the DLQ ARN, never a string.**

```
variable "name"          { type = string }
variable "environment" { type = string }
variable "max_receive_count" {
  type    = number
  default = 3
}
# [...] visibility_timeout, batch_size, schedule_expression
output "queue_url" { value = aws_sqs_queue.main.url }
output "queue_arn" { value = aws_sqs_queue.main.arn }
output "dlq_arn"   { value = aws_sqs_queue.dlq.arn }
```

Each description is the module's public documentation.

```
data "aws_region" "current" {}
data "aws_caller_identity" "current" {}

locals {
  queue_url =
    "https://sqs.${data.aws_region.current.name}.amazonaws.com/${data.aws_caller_identity.current.account_id}/${var.name}-${var.environment}-queue"
}

module "async" {
  source = "../modules/async"
  # [...] name, environment, lambda_*, tunables
}
```

**Deterministic URL breaks the circular module dependency.**



```
resource "aws_lambda_event_source_mapping" "this" {  
    event_source_arn = aws_sqs_queue.main.arn  
    function_name     = var.lambda_function_arn  
    batch_size        = var.batch_size  
}
```

**The ESM is the connector that helps pulling messages  
when they arrive**

```
resource "aws_iam_role_policy" "lambda_sqs" {  
  role    = var.lambda_execution_role_name  
  policy = jsonencode({  
    Statement = [{  
      Effect    = "Allow"  
      Action    = ["sqs:SendMessage", "sqs:ReceiveMessage",  
                  "sqs:DeleteMessage", "sqs:GetQueueAttributes"]  
      Resource  = aws_sqs_queue.main.arn  
    }]  
  })  
}
```

**Four actions, one queue ARN — never a wildcard.**

```
resource "aws_scheduler_schedule" "this" {  
  name = "${var.name}-${var.environment}-schedule"  
  flexible_time_window { mode = "OFF" }  
  schedule_expression      = var.schedule_expression  
  schedule_expression_timezone = var.scheduler_timezone  
  target {  
    arn      = var.lambda_function_arn  
    role_arn = aws_iam_role.scheduler.arn  
  }  
}
```

**Cron invocation, tunable per environment.**

```
resource "aws_iam_role" "scheduler" {
  assume_role_policy = jsonencode({
    Statement = [{
      Effect      = "Allow"
      Principal = { Service = "scheduler.amazonaws.com" }
      Action      = "sts:AssumeRole"
    }]
  })
}
# + aws_iam_role_policy "scheduler_invoke":
#   lambda:InvokeFunction on var.lambda_function_arn
```

**Each service gets its own scoped identity.**

```
cd infra
terraform init
terraform plan -var-file=envs/dev/dev.tfvars
terraform apply -var-file=envs/dev/dev.tfvars
terraform output
```

**Expect 6 new resources; outputs give the queue URL.**

### Key Insight

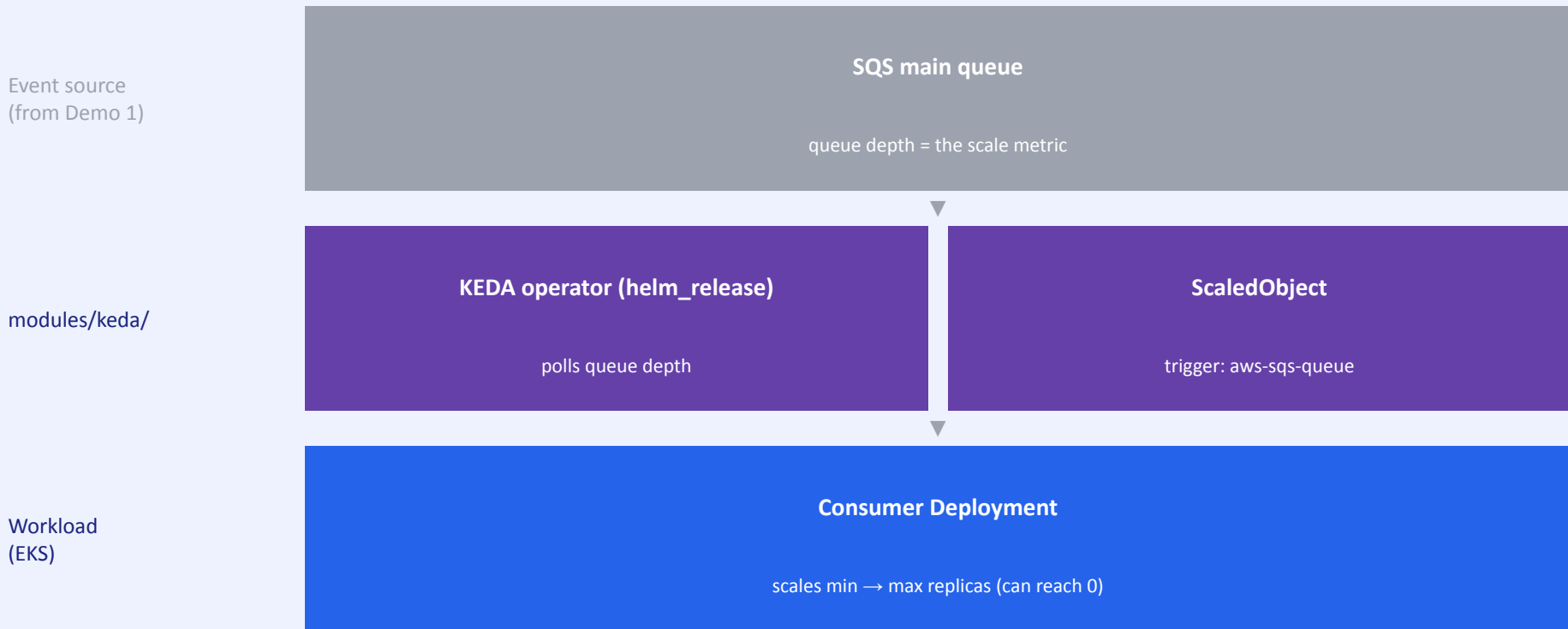
# One Lambda, three triggers — HTTP, queue, schedule.

A job flows `POST /jobs` → HTTP handler returns 202 and enqueues → SQS holds it → the ESM polls and invokes the SQS handler, which writes `results/{job_id}.json` to S3. After `max_receive_count` failures the message lands in the DLQ. Separately, the scheduler invokes the same function with `{"source": "scheduler"}` to run cleanup. The dispatcher routes each event by shape — the module boundary hides which cloud you're on.

# Block 1.5 — KEDA

Event-driven autoscaling for EKS teams (concept only)

## Block 1.5 — How KEDA Scales the Consumer



*The ScaledObject watches the Demo 1 queue and scales the Deployment 0 → N — same queue contract, the consumer is a Pod instead of a Lambda ESM. Concept only tonight; no live demo.*



## KEDA — Kubernetes Event-Driven Autoscaler

KEDA scales Kubernetes Deployments from external event sources — SQS queue depth, Pub/Sub lag, Kafka, and many more — not just CPU and memory. It installs as a cluster add-on and reads metrics no built-in autoscaler can see.

### What it does

Scales on external event metrics, not just CPU/memory

Sources: SQS depth, Pub/Sub lag, Kafka, Prometheus, ...

A poll-based consumer can sit at zero replicas when idle

### Why EKS teams care

Scales up the moment the queue has work

Scales back toward zero when drained — no idle cost

Installed as a Terraform module via `helm_release`

## ScaledObject — Wiring KEDA to the Queue

A ScaledObject is the KEDA resource that binds one Deployment to one event source. Here it targets the consumer Deployment and its trigger watches the SQS queue built in Demo 1 — the queue contract is identical, only the consumer changes.

### It watches the queue you built

Trigger type `aws-sqs-queue`, pointed at the Demo 1 queue URL

Queue grows → scale the Deployment up

Queue drains → scale back down toward min

### Same pattern, different compute

Demo 1's consumer was a Lambda Event Source Mapping

EKS teams' consumer is a Pod in a Deployment

`queue_length_trigger` sets the messages-per-replica target

## Delivery 4 — Deliverable F (EKS Track)

EKS-track teams complete Deliverable F by installing KEDA as a Terraform module and defining a ScaledObject against their Delivery A queue. The D4 spec is the authoritative source for the exact requirements.

### What you deliver

Install KEDA via helm\_release in infra/modules/keda/

Define a ScaledObject targeting the Delivery A queue

Variables: queue\_url, aws\_region, min/max\_replica\_count, queue\_length\_trigger

### Where to learn more

[keda.sh/docs](https://keda.sh/docs) — concepts + the SQS scaler reference

Terraform helm\_release resource docs

D4 Deliverable F spec — authoritative requirements

# Exercise

## 1

Build a Reusable Queue  
Module

Build the `infra/modules/queue` module that the provided root `main.tf` already calls — a main SQS queue plus its dead-letter queue, returned from a single module call.

1. Copy the provided stub files exactly (`provider.tf`, `main.tf`, `variables.tf`) — do not modify them.
2. Create `modules/queue` with `main.tf`, `variables.tf`, `outputs.tf`.
3. Define the DLQ, then the main queue with a `redrive_policy` referencing the DLQ ARN.
4. Expose `queue_url`, `queue_arn`, `dlq_url`, `dlq_arn` as outputs.
5. `terraform plan`, then `terraform apply`.

Targets Delivery 4 — Deliverable A. Standalone: use a fresh repo, not the project repo.

Evidence: terraform output (screenshot or saved text).

# Demo 2

Full CD Pipeline — plan artifacts, environment gates, drift

## Demo 2 — What We're Building



*The apply jobs consume the uploaded plan artifact — apply applies the exact plan the reviewer saw, never a re-plan against changed state.*

## Multi-Env Layout + Partial Backend (Pattern A)

One codebase, one empty backend block, N backend config files. Each environment owns a directory under `infra/envs/` holding its own `*.tfvars` and `backend-*.hcl` — values are supplied at terraform init time, never hardcoded.

### Directory contract

`infra/envs/dev/` and `infra/envs/staging/` — never mixed

Each holds its own `tfvars` and `backend-<env>.hcl`

Scales to 5 environments with no naming collisions

### Backend isolation

backend "s3" {} is empty — filled by `-backend-config`

Separate key per env: `dev/...` vs `staging/...` state files

A destroy in dev cannot touch staging state

## Named Checks + Plan Artifact

PR validation is split into three jobs — terraform fmt, terraform validate, terraform plan — so each reports as its own status check. The plan job freezes the changes into a binary file and preserves it as an artifact for the apply job.

### Three named checks

GitHub reports required checks at the job level

Three jobs = three individually-selectable check names

validate uses `-backend=false` — no credentials needed

### The plan artifact

`terraform plan -out=tfplan` captures the exact change set

`actions/upload-artifact` preserves it across jobs in the run

The plan job also posts the plan as a PR comment



## GitHub Environments — Deployment Gates

An environment: key on a job records the deployment and can require a reviewer to approve before any step runs. This is how a pipeline promotes from dev to staging behind a human gate without scripting approvals by hand.

### dev — no gate

environment: dev records history, no required reviewer

Every merge to main auto-deploys to dev

plan → upload → apply tfplan in one job

### staging — reviewer gate

needs: [apply-dev] — runs only after dev succeeds

GitHub pauses before any step until a reviewer approves

Reviewer sees the plan generated immediately before the pause

## Branch Protection Ruleset on main

A ruleset is GitHub's successor to branch protection rules. Set to Active, it enforces — immediately — that every change to main arrives through a PR that passed the three named checks, on an up-to-date branch.

### Rules enabled

Require a pull request before merging

Require status checks: fmt, validate, plan (exact names)

Require up-to-date · block force-push · restrict deletions

### The name-match gotcha

Checks resolve by the exact string Actions reports

A typo means the check never reports — gate silently passes

Match each ruleset check name to the job name  
character-for-character

► **LIVE DEMO**

**Demo 2**

# **Full CD Pipeline**

```
mkdir -p infra/envs/dev infra/envs/staging
```

**One directory per environment — never mixed.**

```
environment = "dev"  
name        = "file-processor"  
memory_size = 128  
  
max_receive_count      = 3  
visibility_timeout_seconds = 30  
schedule_expression    = "rate(5 minutes)"
```

**Dev is lean — iterate fast and cheap.**

```
environment = "staging"  
name        = "file-processor"  
memory_size = 256  
  
max_receive_count      = 5  
visibility_timeout_seconds = 60  
schedule_expression    = "rate(15 minutes)"
```

**Three-plus values differ so staging catches config bugs.**

```
# infra/envs/dev/backend-dev.hcl
key = "dev/terraform.tfstate"
# infra/envs/staging/backend-staging.hcl
key = "staging/terraform.tfstate"

# infra/provider.tf
terraform {
  backend "s3" {}    # empty - values from -backend-config
}
```

**Empty backend block; per-env state keys isolate destroys.**

```
cd infra  
terraform init -backend-config=envs/dev/backend-dev.hcl  
terraform plan -var-file=envs/dev/dev.tfvars -out=tfplan
```

**The binary plan file is the artifact.**



```
jobs:
  fmt:
    name: terraform fmt          # check #1
    # fmt -check -recursive
  validate:
    name: terraform validate     # check #2
    # init -backend=false; validate
  plan:
    name: terraform plan         # check #3
    # plan -out=tfplan; upload-artifact; PR comment
```

**Three jobs = three required-checkable statuses.**

```
Settings → Rules → Rulesets → New branch ruleset
```

```
Enforcement: Active      Target: main
```

```
[x] Require a pull request before merging
```

```
[x] Require status checks:
```

```
    terraform fmt
```

```
    terraform validate
```

```
    terraform plan
```

```
[x] Require branches to be up to date
```

```
[x] Block force pushes
```

```
[x] Restrict deletions
```

**Check names must match job names exactly — typo =  
silent pass.**

```
apply-dev:
  if: github.event_name == 'push'
      && github.ref == 'refs/heads/main'
  environment: dev          # no required reviewer
  steps:
    # init -backend-config=envs/dev/backend-dev.hcl
    # plan -var-file=envs/dev/dev.tfvars -out=tfplan
    # upload-artifact tfplan
    # apply -auto-approve tfplan
```

**Dev auto-deploys; apply uses the just-made plan.**

```
apply-staging:
  needs: [apply-dev]
  environment: staging      # required reviewer
  steps:
    # init -backend-config=envs/staging/backend-staging.hcl
    # plan -var-file=envs/staging/staging.tfvars
    #   -out=tfplan-staging
    # upload-artifact tfplan-staging
    # apply -auto-approve tfplan-staging
```

**Reviewer approves the exact staged plan before apply.**

```
on:
  workflow_dispatch:
    inputs:
      environment:
        type: choice
        options: [dev, staging]
jobs:
  destroy:
    environment: ${ github.event.inputs.environment }
    # init backend-<env>; destroy -var-file=... -auto-approve
```

**Two gates: manual trigger + reviewer approval.**

```
on:
  schedule:
    - cron: "0 8 * * 1-5"
jobs:
  drift-dev:
    # plan -detailed-exitcode | tee drift_output.txt
    # echo exit_code=${PIPESTATUS[0]} >> $GITHUB_OUTPUT
    # cat drift_output.txt >> $GITHUB_STEP_SUMMARY
    # if exit_code == '2' -> exit 1
  drift-staging:
    # same against staging backend + tfvars
```

**Exit code 2 = drift; daily check catches manual changes.**

### Key Insight

# Apply the plan the reviewer saw — not a re-plan.

terraform plan -out=tfplan freezes the change set into a binary; upload-artifact carries it to the apply job; apply tfplan applies exactly that — and aborts if state drifted since. Re-planning in the apply job, or dropping -out, breaks the guarantee. The branch ruleset completes the picture: no change reaches main without passing all three checks, and no apply runs against a plan the reviewer didn't see.

# Exercise

## 2

Multi-Env Layout +  
Environment Promotion

Evolve a working single-job terraform-cd.yml into a production-grade pipeline that reports each validation step individually, uploads the plan as an artifact, and gates dev → staging behind a required reviewer.

1. Copy the provided stub files exactly — only `.github/workflows/terraform-cd.yml` is yours to rewrite.
2. Split PR validation into three named jobs: `terraform fmt / validate / plan`.
3. The plan job uploads `tfplan` as an artifact and posts it as a PR comment.
4. `apply-dev` (no gate) and `apply-staging` (`environment: staging, required reviewer, needs: apply-dev`).
5. Configure a required reviewer on the staging environment.

Targets Delivery 4 — Deliverable D (core). Standalone: provided stub repo with a working CI baseline.

Evidence: link to the PR where the plan-artifact workflow ran and posted a comment.